

Models

RPI

API Documentation

▼

`pydantic.main.BaseModel`

One of the primary ways of defining schema in Pydantic is via models. Models are simply classes which inherit from `BaseModel` and define fields as annotated attributes.

You can think of models as similar to structs in languages like C, or as the requirements of a single endpoint in an API.

Models share many similarities with Python's **dataclasses**, but have been designed with some subtle-yet-important differences that streamline certain workflows related to validation, serialization, and JSON schema generation. You can find more discussion of this in the **Dataclasses** section of the docs.

Untrusted data can be passed to a model and, after parsing and validation, Pydantic guarantees that the fields of the resultant model instance will conform to the field types defined on the model.

Validation — a *deliberate* misnomer

TL;DR

We use the term "validation" to refer to the process of instantiating a model (or other type) that adheres to specified types and constraints. This task, which Pydantic is well known for, is most widely recognized as "validation" in colloquial terms, even though in other contexts the term "validation" may be more restrictive.

The long version

The potential confusion around the term "validation" arises from the fact that, strictly speaking, Pydantic's primary focus doesn't align precisely with the dictionary definition of "validation":

validation

noun the action of checking or proving the validity or accuracy of something.

In Pydantic, the term "validation" refers to the process of instantiating a model (or other type) that adheres to specified types and constraints. Pydantic guarantees the types and constraints of the output, not the input data. This distinction becomes apparent when considering that Pydantic's `ValidationError` is raised when data cannot be successfully parsed into a model instance.

While this distinction may initially seem subtle, it holds practical significance. In some cases, "validation" goes beyond just model creation, and can include the copying and coercion of data. This can involve copying arguments passed to the constructor in order to perform coercion to a new type without mutating the original input data. For a more in-depth understanding of the implications for your usage, refer to the **Data Conversion** and **Attribute Copies** sections below.

In essence, Pydantic's primary goal is to assure that the resulting structure post-processing (termed "validation") precisely conforms to the applied type hints. Given the widespread adoption of "validation" as the colloquial term for this process, we will consistently use it in our documentation.

While the terms "parse" and "validation" were previously used interchangeably, moving forward, we aim to exclusively employ "validate", with "parse" reserved specifically for discussions related to **JSON parsing**.

Basic model usage

Note

Pydantic relies heavily on the existing Python typing constructs to define models. If you are not familiar with those, the following resources can be useful:

- The **Type System Guides**
- The **mypy documentation**

```
from pydantic import BaseModel, ConfigDict

class User(BaseModel):
    id: int
    name: str = 'Jane Doe'

    model_config = ConfigDict(str_max_length=10)
```

- 1 Pydantic models support a variety of **configuration values** (see [here](#) for the available configuration values).

In this example, `User` is a model with two fields:

- `id`, which is an integer (defined using the `int` type) and is required
- `name`, which is a string (defined using the `str` type) and is not required (it has a default value).

The documentation on **types** expands on the supported types.

Fields can be customized in a number of ways using the `Field()` function. See the [documentation on fields](#) for more information.

The model can then be instantiated:

```
user = User(id='123')
```

`user` is an instance of `User`. Initialization of the object will perform all parsing and validation. If no `ValidationError` exception is raised, you know the resulting model instance is valid.

Fields of a model can be accessed as normal attributes of the `user` object:

```
assert user.name == 'Jane Doe'
assert user.id == 123
assert isinstance(user.id, int)
```

- 1 `name` wasn't set when `user` was initialized, so the default value was used. The `model_fields_set` attribute can be inspected to check the field names explicitly set during instantiation.
- 2 Note that the string `'123'` was coerced to an integer and its value is `123`. More details on Pydantic's coercion logic can be found in the [data conversion](#) section.

The model instance can be serialized using the `model_dump()` method:

```
assert user.model_dump() == {'id': 123, 'name': 'Jane Doe'}
```

Calling `dict` on the instance will also provide a dictionary, but nested fields will not be recursively converted into dictionaries. `model_dump()` also provides numerous arguments to customize the serialization result.

By default, models are mutable and field values can be changed through attribute assignment:

```
user.id = 321
assert user.id == 321
```

Warning

When defining your models, watch out for naming collisions between your field name and its type annotation.

For example, the following will not behave as expected and would yield a validation error:

```
from typing import Optional

from pydantic import BaseModel

class Boo(BaseModel):
    int: Optional[int] = None

m = Boo(int=123) # Will fail to validate.
```

Because of how Python evaluates **annotated assignment statements**, the statement is equivalent to `int: None = None`, thus leading to a validation error.

Model methods and properties

The example above only shows the tip of the iceberg of what models can do. Model classes possess the following methods and attributes:

- `model_validate()`: Validates the given object against the Pydantic model. See [Validating data](#).
- `model_validate_json()`: Validates the given JSON data against the Pydantic model. See [Validating data](#).
- `model_construct()`: Creates models without running validation. See [Creating models without validation](#).
- `model_dump()`: Returns a dictionary of the model's fields and values. See [Serialization](#).
- `model_dump_json()`: Returns a JSON string representation of `model_dump()`. See [Serialization](#).
- `model_copy()`: Returns a copy (by default, shallow copy) of the model. See [Model copy](#).

- `model_json_schema()` : Returns a jsonable dictionary representing the model's JSON Schema. See [JSON Schema](#).
- `model_fields` : A mapping between field names and their definitions (`FieldInfo` instances).
- `model_computed_fields` : A mapping between computed field names and their definitions (`ComputedFieldInfo` instances).
- `model_parametrized_name()` : Computes the class name for parametrizations of generic classes.
- `model_post_init()` : Performs additional actions after the model is instantiated and all field validators are applied.
- `model_rebuild()` : Rebuilds the model schema, which also supports building recursive generic models. See [Rebuilding model schema](#).

Model instances possess the following attributes:

- `model_extra` : The extra fields set during validation.
- `model_fields_set` : The set of fields which were explicitly provided when the model was initialized.

Note

See the API documentation of `BaseModel` for the class definition including a full list of methods and attributes.

Tip

See [Changes to `pydantic.BaseModel`](#) in the [Migration Guide](#) for details on changes from Pydantic V1.

Data conversion

Pydantic may cast input data to force it to conform to model field types, and in some cases this may result in a loss of information. For example:

```
from pydantic import BaseModel

class Model(BaseModel):
    a: int
    b: float
    c: str

print(Model(a=3.000, b='2.72', c=b'binary data').model_dump())
#> {'a': 3, 'b': 2.72, 'c': 'binary data'}
```

This is a deliberate decision of Pydantic, and is frequently the most useful approach. See [this issue](#) for a longer discussion on the subject.

Nevertheless, Pydantic provides a [strict mode](#), where no data conversion is performed. Values must be of the same type as the declared field type.

This is also the case for collections. In most cases, you shouldn't make use of abstract container classes and just use a concrete type, such as `list` :

```
from pydantic import BaseModel

class Model(BaseModel):
    items: list[int]

print(Model(items=(1, 2, 3)))
#> items=[1, 2, 3]
```

- 1 In this case, you might be tempted to use the abstract `Sequence` type to allow both lists and tuples. But Pydantic takes care of converting the tuple input to a list, so in most cases this isn't necessary.

Besides, using these abstract types can also lead to [poor validation performance](#), and in general using concrete container types will avoid unnecessary checks.

Extra data

By default, Pydantic models **won't error when you provide extra data**, and these values will simply be ignored:

```
from pydantic import BaseModel

class Model(BaseModel):
    x: int
```

```
m = Model(x=1, y='a')
assert m.model_dump() == {'x': 1}
```

The `extra` configuration value can be used to control this behavior:

```
from pydantic import BaseModel, ConfigDict

class Model(BaseModel):
    x: int

    model_config = ConfigDict(extra='allow')

m = Model(x=1, y='a')
assert m.model_dump() == {'x': 1, 'y': 'a'}
assert m.__pydantic_extra__ == {'y': 'a'}
```

❶ If `extra` was set to `'forbid'`, this would fail.

The configuration can take three values:

- `'ignore'`: Providing extra data is ignored (the default).
- `'forbid'`: Providing extra data is not permitted.
- `'allow'`: Providing extra data is allowed and stored in the `__pydantic_extra__` dictionary attribute. The `__pydantic_extra__` can explicitly be annotated to provide validation for extra fields.

The validation methods (e.g. `model_validate()`) have an optional `extra` argument that will override the `extra` configuration value of the model for that validation call.

For more details, refer to the `extra` API documentation.

Pydantic dataclasses also support extra data (see the [dataclass configuration](#) section).

Nested models

More complex hierarchical data structures can be defined using models themselves as types in annotations.

```
Python 3.9 and above Python 3.10 and above

from typing import Optional

from pydantic import BaseModel

class Foo(BaseModel):
    count: int
    size: Optional[float] = None

class Bar(BaseModel):
    apple: str = 'x'
    banana: str = 'y'

class Spam(BaseModel):
    foo: Foo
    bars: list[Bar]

m = Spam(foo={'count': 4}, bars=[{'apple': 'x1'}, {'apple': 'x2'}])
print(m)
"""
foo=Foo(count=4, size=None) bars=[Bar(apple='x1', banana='y'), Bar(apple='x2', banana='y')]
"""
print(m.model_dump())
"""
{
  'foo': {'count': 4, 'size': None},
  'bars': [{'apple': 'x1', 'banana': 'y'}, {'apple': 'x2', 'banana': 'y'}],
}
"""
```

Self-referencing models are supported. For more details, see the documentation related to [forward annotations](#).

Rebuilding model schema

When you define a model class in your code, Pydantic will analyze the body of the class to collect a variety of information required to perform validation and serialization, gathered in a core schema. Notably, the model's type annotations are evaluated to understand the valid types for each field (more information can be found in the [Architecture](#) documentation). However, it might be the case that annotations refer to symbols not defined when the model class is being created. To circumvent this issue, the `model_rebuild()` method can be used:

```
from pydantic import BaseModel, PydanticUserError

class Foo(BaseModel):
    x: 'Bar'

try:
    Foo.model_json_schema()
except PydanticUserError as e:
    print(e)
    """
    `Foo` is not fully defined; you should define `Bar`, then call `Foo.model_rebuild()`.

    For further information visit https://errors.pydantic.dev/2/u/class-not-fully-defined
    """

class Bar(BaseModel):
    pass

Foo.model_rebuild()
print(Foo.model_json_schema())
"""
{
  '$defs': {'Bar': {'properties': {}, 'title': 'Bar', 'type': 'object'}},
  'properties': {'x': {'$ref': '#/$defs/Bar'}},
  'required': ['x'],
  'title': 'Foo',
  'type': 'object',
}
```

1 `Bar` is not yet defined when the `Foo` class is being created. For this reason, a **forward annotation** is being used.

Pydantic tries to determine when this is necessary automatically and error if it wasn't done, but you may want to call `model_rebuild()` proactively when dealing with recursive models or generics.

In V2, `model_rebuild()` replaced `update_forward_refs()` from V1. There are some slight differences with the new behavior. The biggest change is that when calling `model_rebuild()` on the outermost model, it builds a core schema used for validation of the whole model (nested models and all), so all types at all levels need to be ready before `model_rebuild()` is called.

Validating data

Pydantic can validate data in three different modes: *Python*, *JSON* and *strings*.

The *Python* mode gets used when using:

- The `__init__()` model constructor. Field values must be provided using keyword arguments.
- `model_validate()`: data can be provided either as a dictionary, or as a model instance (by default, instances are assumed to be valid; see the `revalidate_instances` setting). **Arbitrary objects** can also be provided if explicitly enabled.

The *JSON* and *strings* modes can be used with dedicated methods:

- `model_validate_json()`: data is validated as a JSON string or `bytes` object. If your incoming data is a JSON payload, this is generally considered faster (instead of manually parsing the data as a dictionary). Learn more about JSON parsing in the [JSON](#) documentation.
- `model_validate_strings()`: data is validated as a dictionary (can be nested) with string keys and values and validates the data in JSON mode so that said strings can be coerced into the correct types.

Compared to using the model constructor, it is possible to control several validation parameters when using the `model_validate_*` methods (**strictness**, **extra data**, **validation context**, etc.).

Note

Depending on the types and model configuration involved, the *Python* and *JSON* modes may have different validation behavior (e.g. with **strictness**). If you have data coming from a non-

JSON source, but want the same validation behavior and errors you'd get from the *JSON* mode, our recommendation for now is to either dump your data to JSON (e.g. using `json.dumps()`), or use `model_validate_strings()` if the data takes the form of a (potentially nested) dictionary with string keys and values. Progress for this feature can be tracked in [this issue](#).

Python 3.9 and above Python 3.10 and above

```
from datetime import datetime
from typing import Optional

from pydantic import BaseModel, ValidationError

class User(BaseModel):
    id: int
    name: str = 'John Doe'
    signup_ts: Optional[datetime] = None

m = User.model_validate({'id': 123, 'name': 'James'})
print(m)
#> id=123 name='James' signup_ts=None

try:
    m = User.model_validate_json('{"id": 123, "name": 123}')
except ValidationError as e:
    print(e)
    """
    1 validation error for User
    name
      Input should be a valid string [type=string_type, input_value=123, input_type=int]
    """

m = User.model_validate_strings({'id': '123', 'name': 'James'})
print(m)
#> id=123 name='James' signup_ts=None

m = User.model_validate_strings(
    {'id': '123', 'name': 'James', 'signup_ts': '2024-04-01T12:00:00'}
)
print(m)
#> id=123 name='James' signup_ts=datetime.datetime(2024, 4, 1, 12, 0)

try:
    m = User.model_validate_strings(
        {'id': '123', 'name': 'James', 'signup_ts': '2024-04-01'}, strict=True
    )
except ValidationError as e:
    print(e)
    """
    1 validation error for User
    signup_ts
      Input should be a valid datetime, invalid datetime separator, expected `T`, `t`, `_` or space [type=datetime_parsing, input_value='2024-04-01', input_type=str]
    """
```

Creating models without validation

Pydantic also provides the `model_construct()` method, which allows models to be created **without validation**. This can be useful in at least a few cases:

- when working with complex data that is already known to be valid (for performance reasons)
- when one or more of the validator functions are non-idempotent
- when one or more of the validator functions have side effects that you don't want to be triggered.

 **Warning**

`model_construct()` does not do any validation, meaning it can create models which are invalid. **You should only ever use the `model_construct()` method with data which has already been validated, or that you definitely trust.**

 **Note**

In Pydantic V2, the performance gap between validation (either with direct instantiation or the `model_validate*` methods) and `model_construct()` has been narrowed considerably. For simple models, going with validation may even be faster. If you are using `model_construct()` for performance reasons, you may want to profile your use case before assuming it is actually faster.

Note that for **root models**, the root value can be passed to `model_construct()` positionally, instead of using a keyword argument.

Here are some additional notes on the behavior of `model_construct()`:

- When we say "no validation is performed" — this includes converting dictionaries to model instances. So if you have a field referring to a model type, you will need to convert the inner dictionary to a model yourself.
- If you do not pass keyword arguments for fields with defaults, the default values will still be used.
- For models with private attributes, the `__pydantic_private__` dictionary will be populated the same as it would be when creating the model with validation.
- No `__init__` method from the model or any of its parent classes will be called, even when a custom `__init__` method is defined.

On extra data behavior with `model_construct()`

- For models with `extra` set to `'allow'`, data not corresponding to fields will be correctly stored in the `__pydantic_extra__` dictionary and saved to the model's `__dict__` attribute.
- For models with `extra` set to `'ignore'`, data not corresponding to fields will be ignored — that is, not stored in `__pydantic_extra__` or `__dict__` on the instance.
- Unlike when instantiating the model with validation, a call to `model_construct()` with `extra` set to `'forbid'` doesn't raise an error in the presence of data not corresponding to fields. Rather, said input data is simply ignored.

Defining a custom `__init__()`

Pydantic provides a default `__init__()` implementation for Pydantic models, that is called *only* when using the model constructor (and not with the `model_validate_*`() methods). This implementation delegates validation to `pydantic-core`.

However, it is possible to define a custom `__init__()` on your models. In this case, it will be called unconditionally from all the **validation methods**, without performing validation (and so you should call `super().__init__(**kwargs)` in your implementation).

Defining a custom `__init__()` is not recommended, as all the validation parameters (**strictness**, **extra data behavior**, **validation context**) will be lost. If you need to perform actions after the model was initialized, you can make use of **after field** or **model** validators, or define a `model_post_init()` implementation:

```
import logging
from typing import Any

from pydantic import BaseModel

class MyModel(BaseModel):
    id: int

    def model_post_init(self, context: Any) -> None:
        logging.info("Model initialized with id %d", self.id)
```

Error handling

Pydantic will raise a `ValidationError` exception whenever it finds an error in the data it's validating.

A single exception will be raised regardless of the number of errors found, and that validation error will contain information about all of the errors and how they happened.

See **Error Handling** for details on standard and custom errors.

As a demonstration:

```
from pydantic import BaseModel, ValidationError

class Model(BaseModel):
    list_of_ints: list[int]
    a_float: float

data = {
    'list_of_ints': ['1', 2, 'bad'],
    'a_float': 'not a float',
}

try:
    Model(**data)
except ValidationError as e:
    print(e)
    """
    2 validation errors for Model
    list_of_ints.2
      Input should be a valid integer, unable to parse string as an integer [type=int_parsing, input_value='bad', input_type=str]
```

```
a_float
Input should be a valid number, unable to parse string as a number [type=float_parsing, input_value='not a float', input_type=
"""
```

Arbitrary class instances

(Formerly known as "ORM Mode"/ `from_orm()`).

When using the `model_validate()` method, Pydantic can also validate arbitrary objects, by getting attributes on the object corresponding the field names. One common application of this functionality is integration with object-relational mappings (ORMs).

This feature need to be manually enabled, either by setting the `from_attributes` configuration value, or by using the `from_attributes` parameter on `model_validate()` .

The example here uses [SQLAlchemy](#), but the same approach should work for any ORM.

```
from typing import Annotated

from sqlalchemy import ARRAY, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

from pydantic import BaseModel, ConfigDict, StringConstraints


class Base(DeclarativeBase):
    pass


class CompanyOrm(Base):
    __tablename__ = 'companies'

    id: Mapped[int] = mapped_column(primary_key=True, nullable=False)
    public_key: Mapped[str] = mapped_column(
        String(20), index=True, nullable=False, unique=True
    )
    domains: Mapped[list[str]] = mapped_column(ARRAY(String(255)))


class CompanyModel(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    public_key: Annotated[str, StringConstraints(max_length=20)]
    domains: list[Annotated[str, StringConstraints(max_length=255)]]


co_orm = CompanyOrm(
    id=123,
    public_key='foobar',
    domains=['example.com', 'foobar.com'],
)
print(co_orm)
#> <__main__.CompanyOrm object at 0x0123456789ab>
co_model = CompanyModel.model_validate(co_orm)
print(co_model)
#> id=123 public_key='foobar' domains=['example.com', 'foobar.com']
```

Nested attributes

When using attributes to validate models, model instances will be created from both top-level attributes and deeper-nested attributes as appropriate.

Here is an example demonstrating the principle:

```
from pydantic import BaseModel, ConfigDict


class PetCls:
    def __init__(self, *, name: str) -> None:
        self.name = name


class PersonCls:
    def __init__(self, *, name: str, pets: list[PetCls]) -> None:
        self.name = name
        self.pets = pets


class Pet(BaseModel):
    model_config = ConfigDict(from_attributes=True)
```



```
name: str

class Person(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    name: str
    pets: list[Pet]

bones = PetCls(name='Bones')
orion = PetCls(name='Orion')
anna = PersonCls(name='Anna', pets=[bones, orion])
anna_model = Person.model_validate(anna)
print(anna_model)
#> name='Anna' pets=[Pet(name='Bones'), Pet(name='Orion')]
```

Model copy

API API Documentation

pydantic.main.BaseModel.model_copy

The `model_copy()` method allows models to be duplicated (with optional updates), which is particularly useful when working with frozen models.

```
from pydantic import BaseModel

class BarModel(BaseModel):
    whatever: int

class FooBarModel(BaseModel):
    banana: float
    foo: str
    bar: BarModel

m = FooBarModel(banana=3.14, foo='hello', bar={'whatever': 123})

print(m.model_copy(update={'banana': 0}))
#> banana=0 foo='hello' bar=BarModel(whatever=123)

# normal copy gives the same object reference for bar:
print(id(m.bar) == id(m.model_copy().bar))
#> True
# deep copy gives a new object reference for `bar`:
print(id(m.bar) == id(m.model_copy(deep=True).bar))
#> False
```

Generic models

Pydantic supports the creation of generic models to make it easier to reuse a common model structure. Both the new [type parameter syntax](#) (introduced by [PEP 695](#) in Python 3.12) and the old syntax are supported (refer to [the Python documentation](#) for more details).

Here is an example using a generic Pydantic model to create an easily-reused HTTP response payload wrapper:

Python 3.9 and abovePython 3.12 and above (new syntax)

```
from typing import Generic, TypeVar

from pydantic import BaseModel, ValidationError

DataT = TypeVar('DataT')

class DataModel(BaseModel):
    number: int

class Response(BaseModel, Generic[DataT]):
    data: DataT

print(Response[int](data=1))
#> data=1
print(Response[str](data='hello'))
```

```

print(Response[str](data='value'))
#> data='value'
print(Response[str](data='value').model_dump())
#> {'data': 'value'}

data = DataModel(number=1)
print(Response[DataModel](data=data).model_dump())
#> {'data': {'number': 1}}
try:
    Response[int](data='value')
except ValidationError as e:
    print(e)
    """
    1 validation error for Response[int]
    data
      Input should be a valid integer, unable to parse string as an integer [type=int_parsing, input_value='value', input_type=str]
    """

```

- 1 Declare one or more **type variables** to use to parameterize your model.
- 2 Declare a Pydantic model that inherits from `BaseModel` and `typing.Generic` (in this specific order), and add the list of type variables you declared previously as parameters to the `Generic` parent.
- 3 Use the type variables as annotations where you will want to replace them with other types.

Warning

When parametrizing a model with a concrete type, Pydantic **does not** validate that the provided type is **assignable to the type variable** if it has an upper bound.

Any **configuration**, **validation** or **serialization** logic set on the generic model will also be applied to the parametrized classes, in the same way as when inheriting from a model class. Any custom methods or attributes will also be inherited.

Generic models also integrate properly with type checkers, so you get all the type checking you would expect if you were to declare a distinct type for each parametrization.

Note

Internally, Pydantic creates subclasses of the generic model at runtime when the generic model class is parametrized. These classes are cached, so there should be minimal overhead introduced by the use of generics models.

To inherit from a generic model and preserve the fact that it is generic, the subclass must also inherit from `Generic`:

```

from typing import Generic, TypeVar

from pydantic import BaseModel

TypeX = TypeVar('TypeX')

class BaseClass(BaseModel, Generic[TypeX]):
    x: TypeX

class ChildClass(BaseClass[TypeX], Generic[TypeX]):
    pass

# Parametrize `TypeX` with `int`:
print(ChildClass[int](X=1))
#> X=1

```

You can also create a generic subclass of a model that partially or fully replaces the type variables in the superclass:

```

from typing import Generic, TypeVar

from pydantic import BaseModel

TypeX = TypeVar('TypeX')
TypeY = TypeVar('TypeY')
TypeZ = TypeVar('TypeZ')

class BaseClass(BaseModel, Generic[TypeX, TypeY]):
    x: TypeX
    y: TypeY

class ChildClass(BaseClass[int, TypeY], Generic[TypeY, TypeZ]):
    # ...

```

```

z: typeZ

# Parametrize `TypeY` with `str`:

print(ChildClass[str, int](x='1', y='y', z='3'))
#> x=1 y='y' z=3

```

If the name of the concrete subclasses is important, you can also override the default name generation by overriding the `model_parametrized_name()` method:

```

from typing import Any, Generic, TypeVar

from pydantic import BaseModel

DataT = TypeVar('DataT')

class Response(BaseModel, Generic[DataT]):
    data: DataT

    @classmethod
    def model_parametrized_name(cls, params: tuple[type[Any], ...]) -> str:
        return f'{params[0].__name__.title()}Response'

print(repr(Response[int](data=1)))
#> IntResponse(data=1)
print(repr(Response[str](data='a')))
#> StrResponse(data='a')

```

You can use parametrized generic models as types in other models:

```

from typing import Generic, TypeVar

from pydantic import BaseModel

T = TypeVar('T')

class ResponseModel(BaseModel, Generic[T]):
    content: T

class Product(BaseModel):
    name: str
    price: float

class Order(BaseModel):
    id: int
    product: ResponseModel[Product]

product = Product(name='Apple', price=0.5)
response = ResponseModel[Product](content=product)
order = Order(id=1, product=response)
print(repr(order))
"""
Order(id=1, product=ResponseModel[Product](content=Product(name='Apple', price=0.5)))
"""

```

Using the same type variable in nested models allows you to enforce typing relationships at different points in your model:

```

from typing import Generic, TypeVar

from pydantic import BaseModel, ValidationError

T = TypeVar('T')

class InnerT(BaseModel, Generic[T]):
    inner: T

class OuterT(BaseModel, Generic[T]):
    outer: T
    nested: InnerT[T]

nested = InnerT[int](inner=1)
print(OuterT[int](outer=1, nested=nested))

```

```
#> outer=1 nested=InnerT[int](inner=1)
try:
    print(OuterT[int](outer='a', nested=InnerT(inner='a')))
```

+

```
except ValidationError as e:

    print(e)
    """
    2 validation errors for OuterT[int]
    outer
      Input should be a valid integer, unable to parse string as an integer [type=int_parsing, input_value='a', input_type=str]
    nested.inner
      Input should be a valid integer, unable to parse string as an integer [type=int_parsing, input_value='a', input_type=str]
    """
```

1 The `OuterT` model is parametrized with `int`, but the data associated with the the `T` annotations during validation is of type `str`, leading to validation errors.

Warning

While it may not raise an error, we strongly advise against using parametrized generics in `isinstance()` checks.

For example, you should not do `isinstance(my_model, MyGenericModel[int])`. However, it is fine to do `isinstance(my_model, MyGenericModel)` (note that, for standard generics, it would raise an error to do a subclass check with a parameterized generic class).

If you need to perform `isinstance()` checks against parametrized generics, you can do this by subclassing the parametrized generic class:

```
class MyIntModel(MyGenericModel[int]): ...

isinstance(my_model, MyIntModel)
```

Implementation Details

When using nested generic models, Pydantic sometimes performs revalidation in an attempt to produce the most intuitive validation result. Specifically, if you have a field of type `GenericModel[SomeType]` and you validate data like `GenericModel[SomeCompatibleType]` against this field, we will inspect the data, recognize that the input data is sort of a "loose" subclass of `GenericModel`, and revalidate the contained `SomeCompatibleType` data.

This adds some validation overhead, but makes things more intuitive for cases like that shown below.

```
from typing import Any, Generic, TypeVar

from pydantic import BaseModel

T = TypeVar('T')

class GenericModel(BaseModel, Generic[T]):
    a: T

class Model(BaseModel):
    inner: GenericModel[Any]

print(repr(Model.model_validate(Model(inner=GenericModel[int](a=1))))
#> Model(inner=GenericModel[Any](a=1))
```

Note, validation will still fail if you, for example are validating against `GenericModel[int]` and pass in an instance `GenericModel[str](a='not an int')`.

It's also worth noting that this pattern will re-trigger any custom validation as well, like additional model validators and the like. Validators will be called once on the first pass, validating directly against `GenericModel[Any]`. That validation fails, as `GenericModel[int]` is not a subclass of `GenericModel[Any]`. This relates to the warning above about the complications of using parametrized generics in `isinstance()` and `issubclass()` checks. Then, the validators will be called again on the second pass, during more lax force-revalidation phase, which succeeds. To better understand this consequence, see below:

```
from typing import Any, Generic, Self, TypeVar

from pydantic import BaseModel, model_validator

T = TypeVar('T')

class GenericModel(BaseModel, Generic[T]):
    a: T

    @model_validator(mode='after')
    def validate_after(self: Self) -> Self:
        print('after validator running custom validation...')
        return self

class Model(BaseModel):
    inner: GenericModel[Any]

m = Model.model_validate(Model(inner=GenericModel[int](a=1)))
#> after validator running custom validation...
#> after validator running custom validation...
print(repr(m))
#> Model(inner=GenericModel[Any](a=1))
```

Validation of unparametrized type variables

Validation of unparametrized type variables

When leaving type variables unparametrized, Pydantic treats generic models similarly to how it treats built-in generic types like `list` and `dict`:

- If the type variable is **bound** or **constrained** to a specific type, it will be used.
- If the type variable has a default type (as specified by [PEP 696](#)), it will be used.
- For unbound or unconstrained type variables, Pydantic will fallback to `Any`.

```
from typing import Generic

from typing_extensions import TypeVar

from pydantic import BaseModel, ValidationError

T = TypeVar('T')
U = TypeVar('U', bound=int)
V = TypeVar('V', default=str)

class Model(BaseModel, Generic[T, U, V]):
    t: T
    u: U
    v: V

print(Model(t='t', u=1, v='v'))
#> t='t' u=1 v='v'

try:
    Model(t='t', u='u', v=1)
except ValidationError as exc:
    print(exc)
    """
    2 validation errors for Model
    u
      Input should be a valid integer, unable to parse string as an integer [type=int_parsing, input_value='u', input_type=str]
    v
      Input should be a valid string [type=string_type, input_value=1, input_type=int]
    """
```

 **Warning**

In some cases, validation against an unparametrized generic model can lead to data loss. Specifically, if a subtype of the type variable upper bound, constraints, or default is being used and the model isn't explicitly parametrized, the resulting type **will not be** the one being provided:

```
from typing import Generic, TypeVar

from pydantic import BaseModel

ItemT = TypeVar('ItemT', bound='ItemBase')

class ItemBase(BaseModel): ...

class IntItem(ItemBase):
    value: int

class ItemHolder(BaseModel, Generic[ItemT]):
    item: ItemT

loaded_data = {'item': {'value': 1}}

print(ItemHolder(**loaded_data))
#> item=ItemBase()

print(ItemHolder[IntItem](**loaded_data))
#> item=IntItem(value=1)
```

1

 When the generic isn't parametrized, the input data is validated against the `ItemT` upper bound. Given that `ItemBase` has no fields, the `item` field information is lost.

2

 In this case, the type variable is explicitly parametrized, so the input data is validated against the `IntItem` class.

Serialization of unparametrized type variables

The behavior of serialization differs when using type variables with **upper bounds**, **constraints**, or a default value:

If a Pydantic model is used in a type variable upper bound and the type variable is never parametrized, then Pydantic will use the upper bound for validation but treat the value as `Any` in terms of serialization:

```
from typing import Generic, TypeVar
```

```

from pydantic import BaseModel

class ErrorDetails(BaseModel):
    foo: str

ErrorDataT = TypeVar('ErrorDataT', bound=ErrorDetails)

class Error(BaseModel, Generic[ErrorDataT]):
    message: str
    details: ErrorDataT

class MyErrorDetails(ErrorDetails):
    bar: str

# serialized as Any
error = Error(
    message='We just had an error',
    details=MyErrorDetails(foo='var', bar='var2'),
)
assert error.model_dump() == {
    'message': 'We just had an error',
    'details': {
        'foo': 'var',
        'bar': 'var2',
    },
}

# serialized using the concrete parametrization
# note that ``bar': 'var2`` is missing
error = Error[ErrorDetails](
    message='We just had an error',
    details=ErrorDetails(foo='var'),
)
assert error.model_dump() == {
    'message': 'We just had an error',
    'details': {
        'foo': 'var',
    },
}

```

Here's another example of the above behavior, enumerating all permutations regarding bound specification and generic type parametrization:

```

from typing import Generic, TypeVar

from pydantic import BaseModel

TBound = TypeVar('TBound', bound=BaseModel)
TNoBound = TypeVar('TNoBound')

class IntValue(BaseModel):
    value: int

class ItemBound(BaseModel, Generic[TBound]):
    item: TBound

class ItemNoBound(BaseModel, Generic[TNoBound]):
    item: TNoBound

item_bound_inferred = ItemBound(item=IntValue(value=3))
item_bound_explicit = ItemBound[IntValue](item=IntValue(value=3))
item_no_bound_inferred = ItemNoBound(item=IntValue(value=3))
item_no_bound_explicit = ItemNoBound[IntValue](item=IntValue(value=3))

# calling `print(x.model_dump())` on any of the above instances results in the following:
#> {'item': {'value': 3}}

```

However, if **constraints** or a default value (as per **PEP 696**) is being used, then the default type or constraints will be used for both validation and serialization if the type variable is not parametrized. You can override this behavior using **SerializeAsAny**:

Python 3.9 and above **Python 3.13 and above**

```

from typing import Generic

```

```

from typing_extensions import TypedDict

from pydantic import BaseModel, SerializeAsAny

class ErrorDetails(BaseModel):
    foo: str

ErrorDataT = TypedDict('ErrorDataT', default=ErrorDetails)

class Error(BaseModel, Generic[ErrorDataT]):
    message: str
    details: ErrorDataT

class MyErrorDetails(ErrorDetails):
    bar: str

# serialized using the default's serializer
error = Error(
    message='We just had an error',
    details=MyErrorDetails(foo='var', bar='var2'),
)
assert error.model_dump() == {
    'message': 'We just had an error',
    'details': {
        'foo': 'var',
    },
}
# If `ErrorDataT` was using an upper bound, `bar` would be present in `details`.

class SerializeAsAnyError(BaseModel, Generic[ErrorDataT]):
    message: str
    details: SerializeAsAny[ErrorDataT]

# serialized as Any
error = SerializeAsAnyError(
    message='We just had an error',
    details=MyErrorDetails(foo='var', bar='baz'),
)
assert error.model_dump() == {
    'message': 'We just had an error',
    'details': {
        'foo': 'var',
        'bar': 'baz',
    },
}

```

Dynamic model creation

API API Documentation

pydantic.main.create_model

There are some occasions where it is desirable to create a model using runtime information to specify the fields. Pydantic provides the `create_model()` function to allow models to be created dynamically:

```

from pydantic import BaseModel, create_model

DynamicFoobarModel = create_model('DynamicFoobarModel', foo=str, bar=(int, 123))

# Equivalent to:

class StaticFoobarModel(BaseModel):
    foo: str
    bar: int = 123

```

Field definitions are specified as keyword arguments, and should either be:

- A single element, representing the type annotation of the field.
- A two-tuple, the first element being the type and the second element the assigned value (either a default or the `Field()` function).

Here is a more advanced example:

```
from typing import Annotated

from pydantic import BaseModel, Field, PrivateAttr, create_model

DynamicModel = create_model(
    'DynamicModel',
    foo=(str, Field(alias='F00')),
    bar=Annotated[str, Field(description='Bar field')],
    _private=(int, PrivateAttr(default=1)),
)

class StaticModel(BaseModel):
    foo: str = Field(alias='F00')
    bar: Annotated[str, Field(description='Bar field')]
    _private: int = PrivateAttr(default=1)
```

The special keyword arguments `__config__` and `__base__` can be used to customize the new model. This includes extending a base model with extra fields.

```
from pydantic import BaseModel, create_model

class FooModel(BaseModel):
    foo: str
    bar: int = 123

BarModel = create_model(
    'BarModel',
    apple=(str, 'russet'),
    banana=(str, 'yellow'),
    __base__=FooModel,
)
print(BarModel)
#> <class '__main__.BarModel'>
print(BarModel.model_fields.keys())
#> dict_keys(['foo', 'bar', 'apple', 'banana'])
```

You can also add validators by passing a dictionary to the `__validators__` argument.

```
from pydantic import ValidationError, create_model, field_validator

def alphanum(cls, v):
    assert v.isalnum(), 'must be alphanumeric'
    return v

validators = {
    'username_validator': field_validator('username')(alphanum)
}

UserModel = create_model(
    'UserModel', username=(str, ...), __validators__=validators
)

user = UserModel(username='scolvin')
print(user)
#> username='scolvin'

try:
    UserModel(username='scolvi%n')
except ValidationError as e:
    print(e)
    """
    1 validation error for UserModel
    username
      Assertion failed, must be alphanumeric [type=assertion_error, input_value='scolvi%n', input_type=str]
    """
```

- 1 Make sure that the validators names do not clash with any of the field names as internally, Pydantic gathers all members into a namespace and mimics the normal creation of a class using the `types module utilities`.

 Note

To pickle a dynamically created model:

- the model must be defined globally
- the `__module__` argument must be provided

 Warning

This function may execute arbitrary code contained in field annotations, if string references need to be evaluated.

See [Security implications of introspecting annotations](#) for more information.

See also: the [dynamic model example](#), providing guidelines to derive an optional model from another one.

RootModel and custom root types

API Documentation

`pydantic.root_model.RootModel`

Pydantic models can be defined with a "custom root type" by subclassing `pydantic.RootModel`.

The root type can be any type supported by Pydantic, and is specified by the generic parameter to `RootModel`. The root value can be passed to the model `__init__` or `model_validate` via the first and only argument.

Here's an example of how this works:

```
from pydantic import RootModel

Pets = RootModel[list[str]]
PetsByName = RootModel[dict[str, str]]

print(Pets(['dog', 'cat']))
#> root=['dog', 'cat']
print(Pets(['dog', 'cat']).model_dump_json())
#> ["dog","cat"]
print(Pets.model_validate(['dog', 'cat']))
#> root=['dog', 'cat']
print(Pets.model_json_schema())
"""
{'items': {'type': 'string'}, 'title': 'RootModel[list[str]]', 'type': 'array'}
"""

print(PetsByName({'Otis': 'dog', 'Milo': 'cat'}))
#> root={'Otis': 'dog', 'Milo': 'cat'}
print(PetsByName({'Otis': 'dog', 'Milo': 'cat'}).model_dump_json())
#> {"Otis":"dog","Milo":"cat"}
print(PetsByName.model_validate({'Otis': 'dog', 'Milo': 'cat'}))
#> root={'Otis': 'dog', 'Milo': 'cat'}
```

If you want to access items in the `root` field directly or to iterate over the items, you can implement custom `__iter__` and `__getitem__` functions, as shown in the following example.

```
from pydantic import RootModel

class Pets(RootModel):
    root: list[str]

    def __iter__(self):
        return iter(self.root)

    def __getitem__(self, item):
        return self.root[item]

pets = Pets.model_validate(['dog', 'cat'])
print(pets[0])
#> dog
print(next for pet in pets)
```

```
print([pet for pet in pets],  
#> ['dog', 'cat'])
```

You can also create subclasses of the parametrized root model directly:

```
from pydantic import RootModel  
  
class Pets(RootModel[list[str]]):  
    def describe(self) -> str:  
        return f'Pets: {" ".join(self.root)}'  
  
my_pets = Pets.model_validate(['dog', 'cat'])  
  
print(my_pets.describe())  
#> Pets: dog, cat
```

Faux immutability

Models can be configured to be immutable via `model_config['frozen'] = True`. When this is set, attempting to change the values of instance attributes will raise errors. See the [API reference](#) for more details.

Note

This behavior was achieved in Pydantic V1 via the config setting `allow_mutation = False`. This config flag is deprecated in Pydantic V2, and has been replaced with `frozen`.

Warning

In Python, immutability is not enforced. Developers have the ability to modify objects that are conventionally considered "immutable" if they choose to do so.

```
from pydantic import BaseModel, ConfigDict, ValidationError  
  
class FooBarModel(BaseModel):  
    model_config = ConfigDict(frozen=True)  
  
    a: str  
    b: dict  
  
foobar = FooBarModel(a='hello', b={'apple': 'pear'})  
  
try:  
    foobar.a = 'different'  
except ValidationError as e:  
    print(e)  
    """  
    1 validation error for FooBarModel  
    a  
      Instance is frozen [type=frozen_instance, input_value='different', input_type=str]  
    """  
  
print(foobar.a)  
#> hello  
print(foobar.b)  
#> {'apple': 'pear'}  
foobar.b['apple'] = 'grape'  
print(foobar.b)  
#> {'apple': 'grape'}
```

Trying to change `a` caused an error, and `a` remains unchanged. However, the dict `b` is mutable, and the immutability of `foobar` doesn't stop `b` from being changed.

Abstract base classes

Pydantic models can be used alongside Python's [Abstract Base Classes](#) (ABCs).

```
import abc  
  
from pydantic import BaseModel  
  
class FooBarModel(BaseModel, abc.ABC):  
    a: str
```

```
b: int

@abc.abstractmethod

def my_abstract_method(self):
    pass
```

Field ordering

Field order affects models in the following ways:

- field order is preserved in the model [JSON Schema](#)
- field order is preserved in [validation errors](#)
- field order is preserved when [serializing data](#)

```
from pydantic import BaseModel, ValidationError

class Model(BaseModel):
    a: int
    b: int = 2
    c: int = 1
    d: int = 0
    e: float

print(Model.model_fields.keys())
#> dict_keys(['a', 'b', 'c', 'd', 'e'])
m = Model(e=2, a=1)
print(m.model_dump())
#> {'a': 1, 'b': 2, 'c': 1, 'd': 0, 'e': 2.0}
try:
    Model(a='x', b='x', c='x', d='x', e='x')
except ValidationError as err:
    error_locations = [e['loc'] for e in err.errors()]

print(error_locations)
#> [('a',), ('b',), ('c',), ('d',), ('e',)]
```

Automatically excluded attributes

Class variables

Attributes annotated with [ClassVar](#) are properly treated by Pydantic as class variables, and will not become fields on model instances:

```
from typing import ClassVar

from pydantic import BaseModel

class Model(BaseModel):
    x: ClassVar[int] = 1

    y: int = 2

m = Model()
print(m)
#> y=2
print(Model.x)
#> 1
```

Private model attributes

[API](#) [API Documentation](#)

`pydantic.fields.PrivateAttr`

Attributes whose name has a leading underscore are not treated as fields by Pydantic, and are not included in the model schema. Instead, these are converted into a "private attribute" which is not validated or even set during calls to `__init__`, `model_validate`, etc.

Here is an example of usage:

```

from datetime import datetime
from random import randint
from typing import Any

from pydantic import BaseModel, PrivateAttr

class TimeAwareModel(BaseModel):
    _processed_at: datetime = PrivateAttr(default_factory=datetime.now)
    _secret_value: str

    def model_post_init(self, context: Any) -> None:
        # this could also be done with `default_factory`:
        self._secret_value = randint(1, 5)

m = TimeAwareModel()
print(m._processed_at)
#> 2032-01-02 03:04:05.000006
print(m._secret_value)
#> 3

```

Private attribute names must start with underscore to prevent conflicts with model fields. However, dunder names (such as `__attr__`) are not supported, and will be completely ignored from the model definition.

Model signature

All Pydantic models will have their signature generated based on their fields:

```

import inspect

from pydantic import BaseModel, Field

class FooModel(BaseModel):
    id: int
    name: str = None
    description: str = 'Foo'
    apple: int = Field(alias='pear')

print(inspect.signature(FooModel))
#> (*, id: int, name: str = None, description: str = 'Foo', pear: int) -> None

```

An accurate signature is useful for introspection purposes and libraries like `FastAPI` or `hypothesis`.

The generated signature will also respect custom `__init__` functions:

```

import inspect

from pydantic import BaseModel

class MyModel(BaseModel):
    id: int
    info: str = 'Foo'

    def __init__(self, id: int = 1, *, bar: str, **data) -> None:
        """My custom init!"""
        super().__init__(id=id, bar=bar, **data)

print(inspect.signature(MyModel))
#> (id: int = 1, *, bar: str, info: str = 'Foo') -> None

```

To be included in the signature, a field's alias or name must be a valid Python identifier. Pydantic will prioritize a field's alias over its name when generating the signature, but may use the field name if the alias is not a valid Python identifier.

If a field's alias and name are *both* not valid identifiers (which may be possible through exotic use of `create_model`), a `**data` argument will be added. In addition, the `**data` argument will always be present in the signature if `model_config['extra'] == 'allow'`.

Structural pattern matching

Pydantic supports structural pattern matching for models, as introduced by [PEP 636](#) in Python 3.10.


```
from pydantic import BaseModel

class Pet(BaseModel):
    name: str
    species: str

a = Pet(name='Bones', species='dog')

match a:
    # match `species` to 'dog', declare and initialize `dog_name`
    case Pet(species='dog', name=dog_name):
        print(f'{dog_name} is a dog')
#> Bones is a dog
    # default case
    case _:
        print('No dog matched')
```

Note

A match-case statement may seem as if it creates a new model, but don't be fooled; it is just syntactic sugar for getting an attribute and either comparing it or declaring and initializing it.

Attribute copies

In many cases, arguments passed to the constructor will be copied in order to perform validation and, where necessary, coercion.

In this example, note that the ID of the list changes after the class is constructed because it has been copied during validation:

```
from pydantic import BaseModel

class C1:
    arr = []

    def __init__(self, in_arr):
        self.arr = in_arr

class C2(BaseModel):
    arr: list[int]

arr_orig = [1, 9, 10, 3]

c1 = C1(arr_orig)
c2 = C2(arr=arr_orig)
print(f'{id(c1.arr) == id(c2.arr)=}')
#> id(c1.arr) == id(c2.arr)=False
```

Note

There are some situations where Pydantic does not copy attributes, such as when passing models — we use the model as is. You can override this behaviour by setting `model_config['revalidate_instances'] = 'always'`.

Was this page helpful?

